

Java Exception

Handling

Lab 07

Why Exceptions?

- We seek robust programs
- When something unexpected occurs
 - Ensure program detects the problem
 - Then program must do something about it
- Extensive testing of special situations can result in "spaghetti code"
- We need mechanism to check for problem where it could occur

What is an Exception?

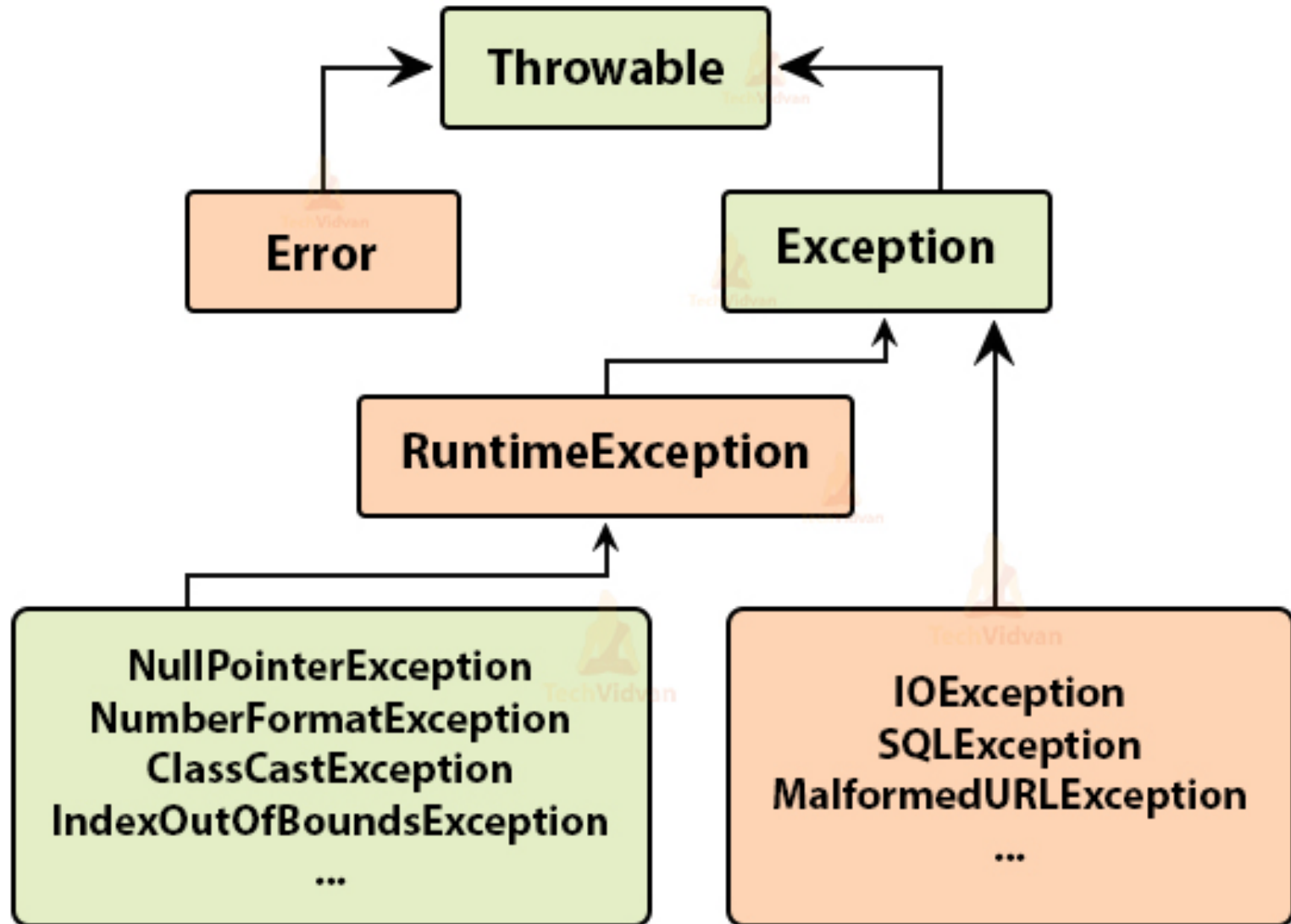
An exception is a class used to store error information.

Exceptions are thrown in exceptional circumstances. If an exception is not caught / handled, the program terminates

Handling Exceptions is one of the most basic and fundamental things a developer should know by heart.

Sadly, this is often overlooked, and the importance of exception handling is underestimated - it's as important as the rest of the code.

Exception Hierarchy in Java



Java Exception Hierarchy

- ▣ Superclass **Throwable**
 - Subclass **Exception**
 - ▣ Exceptional situations
 - ▣ Should be caught by program
 - Subclass **Error**
 - ▣ Examples JVM errors and resource exhaustion
 - ▣ Typically not caught by program
 - ▣ Program can't usually recover from these

Java Exception Hierarchy

▣ Unchecked exceptions

- Indicate programming errors such as precondition violations
- It's your fault. Fix it!
- Don't obey Catch or declare rule.
- Extend **RuntimeException**

▣ Checked exceptions

- Must obey Catch or Declare rule.
- Should be caught by some level of program
- Includes **IOExceptions** and any other exceptions that extend **Exception** but not **RuntimeException**.

Common Runtime Exceptions

java.lang. **NullPointerException**

java.lang. **ArrayIndexOutOfBoundsException**

java.lang. **IndexOutOfBoundsException**

java.lang. **IllegalArgumentException**

java.lang. **ClassCastException**

java.lang. **ArithmeticException**

java.util. **NoSuchElementException**

java.util. **IllegalStateException**

java.io. **ConcurrentModificationException**

Common Checked Exceptions

java.io. **IOException**

java.io. **FileNotFoundException**

java.net. **SocketException**

java.io. **MalformedURLException**

java.io. **UnknownHostException**

java.lang. **InterruptedException**

Errors

java.lang. **OutOfMemoryError**

java.lang. **StackOverflowError**

java.lang. **ThreadDeath** (deprecated)

The Catch / Declare Rule

RuntimeExceptions are unchecked exceptions.

All other Exceptions are checked exceptions.

If your method calls a checked exception, you must either:

- Catch the exception and handle it, or**
- Declare it by adding a throws clause to your method's header.**

Using Exceptions

Reserved words that you need to know to use exception handling ::

try - try this section and see what happens

catch - if the try blew up, catch the exception thrown

finally - this section always happens no matter what

Using Exceptions

```
public static void main(String args[ ])
{
    int num=32;
    if(num==32)
        throw new Exception("num==32");
}
```

This code will not compile. Why not?

Open

exceptionone.java

exceptiontwo.java

Using Exceptions

```
public static void main(String args[ ]) throws Exception
{
    int num=32;
    if(num==32)
        throw new Exception("num==32");
}
```

You have to have **throws** because you are throwing a checked Exception.

Using Exceptions

```
public static void main(String args[ ])
{
    int num=32;
    if(num==32)
        throw new RuntimeException("num==32");
}
```

You DO NOT have to have throws because you are throwing an unchecked Exception.

Open

exceptionthree.java

exceptionfour.java

Using Exceptions

```
try{  
    int num= 3/0;  
}
```

//must have a catch or finally block

System.out.println("compsci");

This will not compile!!!!

What is an Exception?

```
try{  
    int num=3/0;  
}
```

//catch is optional

```
finally{  
    System.out.println("divby0");  
}  
System.out.println("compsci");
```

Catching an exception

```
try { // statement that could throw an exception
    }
catch (<exception type> e) {
    // statements that handle the exception
}
catch (<exception type> e) { //e higher in hierarchy
    // statements that handle the exception
}
finally {
    // release resources
}
//other statements
```

- **At most one catch block executes**
- **finally block always executes once, whether there's an error or not**

Execution of try catch blocks

- For normal execution:
 - try block executes, then finally block executes, then other statements execute
- When an error is caught and the catch block throws an exception or returns:
 - try block is interrupted
 - catch block executes (until throw or return statement)
 - finally block executes
- When error is caught and catch block doesn't throw an exception or return:
 - try block is interrupted
 - catch block executes
 - finally block executes
 - other statements execute
- When an error occurs that is not caught:
 - try block is interrupted
 - finally block executes

Catch processing example

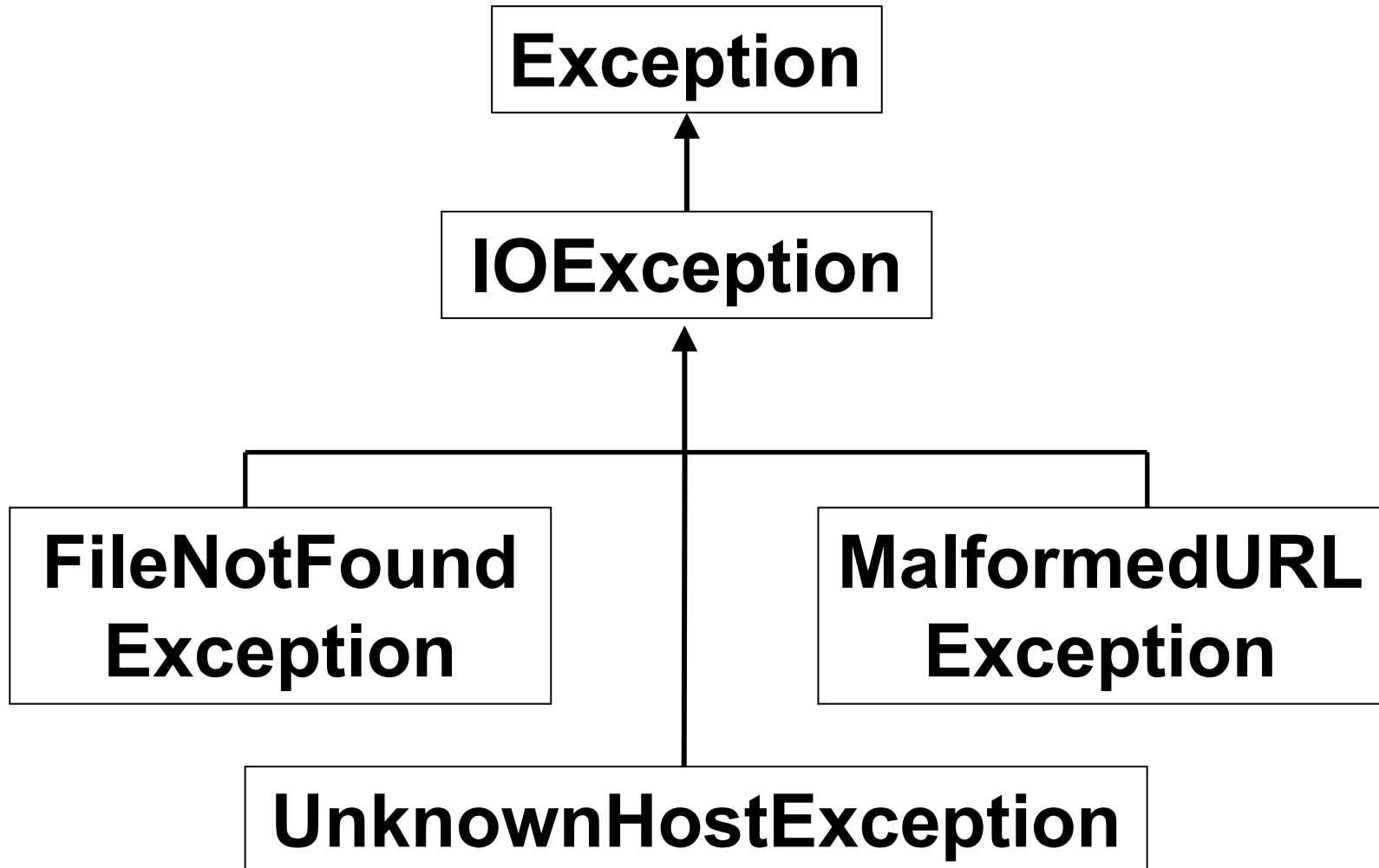
```
print("now");  
try  
{ print("is ");  
  throw new MyException();  
  print("a ");  
}  
catch(MyException e) { print("the "); }  
print("time\n");
```

- Prints "now is the time".
- Note that exceptions don't have to be used only for error handling
- Would it be a good idea to exceptions for non-error processing?
- But any other use is likely to result in code that's hard to understand.

Multiple Catch Blocks

**Catch blocks are
checked for a match
in the order that
they're listed.**

Input / Output Exceptions





Multiple Catch Blocks – ORDER MATTERS!

```
try{
    //code would go here
}
catch(Exception e){
    // Code to handle all other Exceptions
}
catch(IOException e){
    // Code to handle any other IO Exception
}
catch(FileNotFoundException e){
    // Code to handle not finding the specified file
}
finally{
    System.out.println("this always happens");
}
```

Multiple Catch Blocks – ORDER MATTERS!

```
try{
    //code would go here
}
catch(FileNotFoundException e){
    // Code to handle not finding the specified file.
}
catch(IOException e){
    // Code to handle any other IO Exception
}
catch(Exception e){
    // Code to handle all other Exceptions
}
finally{
    System.out.println("this always happens");
}
```

Why not do this?

```
try{
    //code would go here
}
catch(NullPointerException e){
    // Code to handle the lack of a constructor
    // being called.
}
catch(ClassCastException e){
    // Code to handle the fact that your object
    // is the wrong type.
}
finally{
    System.out.println("this always happens");
}
```

A red arrow points from the title 'Why not do this?' to the 'NullPointerException e' catch block. Another red arrow points from the same title to the 'ClassCastException e' catch block.

Runtime exceptions generally shouldn't be caught, because they represent a programming error and should be fixed when you're writing the code.



Why not do this?

```
try{  
    int i=0;  
    while (true) {  
        range[i++] .climb(); }  
}catch (IndexOutOfBoundsException e) {  
  
}
```

**Don't use exceptions for
normal flow control!**



Why not do this?

No!

```
try{
    Iterator i = collection.iterator();
    while (true) {
        Foo foo = (Foo) i.next(); ...
    }
} catch (NoSuchElementException e) {}
```

Yes!

```
Iterator i = collection.iterator();
while (i.hasNext()) {
    Foo foo = (Foo) i.next(); ...
}
```

Don't Ignore Exceptions

Never leave a catch block empty

```
try{  
    . . .  
}catch (Exception e) { }
```

Tempting since an empty catch block won't
raise complaints from compiler.

Don't do this!

Don't Ignore Exceptions

- If its empty, its highly suspect
 - The purpose of exception is to force programmers handle exceptional conditions
- At least have a comment which explains why its ok to ignore this exception



Advantages of Exceptions

- **Separating Error-Handling Code from "Regular" Code**
- **Propagating Errors Up the Call Stack**
- **Grouping and Differentiating Error Types**



Validating Method Arguments

- **Start a method with argument validation**
- **Throw an exception if a test fails**
- **Usually unchecked exceptions are thrown:**
 - **IllegalArgumentException**
 - **NullPointerException**
 - **IllegalStateException**



Validating Method Arguments

Some common validations:

- check for a null reference
- check that text has visible content
- check that a number is in some range

Validating Method Arguments

```
public Constructor (String name, double speed) {  
    if (name == null || name.trim().isEmpty()) {  
        throw new IllegalArgumentException(  
            "Name has no content.");  
    }  
    if (speed < 0.0 || speed > 1.0) {  
        throw new IllegalArgumentException(  
            "Speed not in range [0..1]: " + speed);  
    }  
  
    this.name = name;  
    this.speed = speed;  
}
```

Document Exceptions

Checked Exceptions:

- Declare (required by compiler)
- And Document (with @throws tag in JavaDoc)

Unchecked Exceptions:

- Document (with @throws tag in JavaDoc)

Don't shortcut with superclass exceptions

- Example: Don't catch "Exception" instead of NullPointerException

Ok to document common exceptions at class level

- Example: "All methods in class throw NullPointerException if a null object reference is passed in any parameter"

Open

exceptionfive.java

exceptionsix.java